Import Data and EDA

1. Preprocessing Data

Below are some preprocessing steps to clean raw data for clearly understanding and exploring data.

- Import and read data from excel file to workspace.
- Cleaning blank rows, duplicate rows
- Processing with missing values
- Digitizing continuous data

```
import pandas as pd
import numpy as np
df =pd.read_excel("Database_Business Intelligence Analyst_CME Solar.xlsx", sheet_name="Sheet1")
```

```
# Remove blanks rows
df.drop(df[df["Company Name"].isnull()].index, inplace=True)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 1463 entries, 0 to 1486
Data columns (total 25 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   Company Name                 1463 non-null   float64
 1   Industry (NAICS)             1463 non-null   object
 2   Main Activities (NAICS)      1432 non-null   object
 3   State/County                 1423 non-null   object
 4   Total operating revenue      1463 non-null   float64
 5   Number of Employees          1428 non-null   object
 6   Address                      1432 non-null   object
 7   Email                        157 non-null    object
 8   Phone                        1370 non-null   object
 9   Website                      176 non-null    object
 10  Incorporation Date           1429 non-null   float64
 11  Industry                     1463 non-null   float64
 12  level_2                      1463 non-null   object
 13  level_3                      1463 non-null   object
 14  level_4                      1423 non-null   object
 15  level_5                      942 non-null    object
 16  level_6                      564 non-null    object
 17  Number of factories          1016 non-null   object
 18  Name of factory 1            966 non-null    object
 19  Location factory 1           1118 non-null   object
 20  Rooftop Size factory 1 (m2)  1034 non-null   object
 21  Existing client?             85 non-null     object
 22  Existing solar rooftop?      1026 non-null   object
 23  Region                       1423 non-null   object
 24  Column2                      0 non-null      float64
dtypes: float64(5), object(20)
memory usage: 297.2+ KB
```

```
# Create Lattitude and Longtitude
df['longURL'] = df['Location factory 1'].apply(shortTolongLink)
df['lat'] = df['longURL'].map(getlat)
df['lon'] = df['longURL'].map(getlong)
```

```
# Cleaning Existing client and Existing solar rooftop columns
df['Existing client'] = df['Existing client?'].map(lambda a: 'Yes' if a=='Yes' or a== 'x' else 'No')
df['Existing solar rooftop'] = df['Existing solar rooftop?'].fillna("Missing")
df['Existing solar rooftop'] = df['Existing solar rooftop'].map(lambda a: 'Missing' if a==' ' else a)
```

```
print(df['Existing solar rooftop'].value_counts())
print(df['Existing client'].value_counts())
```

```
No         946
Missing    438
Yes         79
Name: Existing solar rooftop, dtype: int64
No     1432
```

```
Yes      31
Name: Existing client, dtype: int64
```

```
# Create 'No of Employees', 'Year updated' columns
df[['No of Employees', 'Year updated']] = df['Number of Employees'].str.split(' ', expand=True)
df['Year updated'] = df['Year updated'].str.replace('(', '').str.replace(')', '')
df['Year updated'] = df['Year updated'].fillna('1900').astype(int)
```

```
<ipython-input-308-781e66f35ac8>:3: FutureWarning:

The default value of regex will change from True to False in a future version. In addition, single character regular expressions will *
```

```
df.tail(2)
```

	Company Name	Industry (NAICS)	Main Activities (NAICS)	State/County	Total operating revenue	Number of Employees	Address	Email	Phone	Website	...	Existing solar rooftop?	
1485	182862.0	Glass Product Manufacturing Made of Purchased ...	All Other Miscellaneous Chemical Product and P...	South Eastern Vietnam	834751.34	20 (2020)	21/24 Tran Dinh Xu, Cau Kho Ward,	NaN	(84)-2862515280	NaN	...	NaN	S
1486	182866.0	Iron and Steel Mills and Ferroalloy Manufactur...	Iron and Steel Mills and Ferroalloy Manufactur...	Red River Delta	833712.24	320 (2020)	Group 14, Dong Anh Township,	NaN	(84)-2438833649	NaN	...	NaN	I

2 rows × 32 columns

```
df2 = df.drop(columns= ['Existing client?', 'Existing solar rooftop?', 'longURL', 'Column2', 'Number of Employees'])
df2 = df2[df2['Company Name'] != 179030.0]
df2['Company Name']  = df2['Company Name'].astype('string')
df2['No of Employees'] = df2['No of Employees'].str.replace(',','')
df2['No of Employees'] = df2['No of Employees'].fillna(0)
df2['No of Employees'] = df2['No of Employees'].astype(int)
```

```
# Clean ['Rooftop Size factory 1 (m2)']

df2['Rooftop Size factory 1 (m2)'] = df2['Rooftop Size factory 1 (m2)'].replace('?', 0)
df2['Rooftop Size factory 1 (m2)'] = df2['Rooftop Size factory 1 (m2)'].replace('(', '').replace(')', '')
df2['Rooftop Size factory 1 (m2)'].info()
```

```
<class 'pandas.core.series.Series'>
Int64Index: 1462 entries, 0 to 1486
Series name: Rooftop Size factory 1 (m2)
Non-Null Count  Dtype
--------------  -----
1033 non-null   float64
dtypes: float64(1)
memory usage: 22.8 KB
```

2. Ploting Data

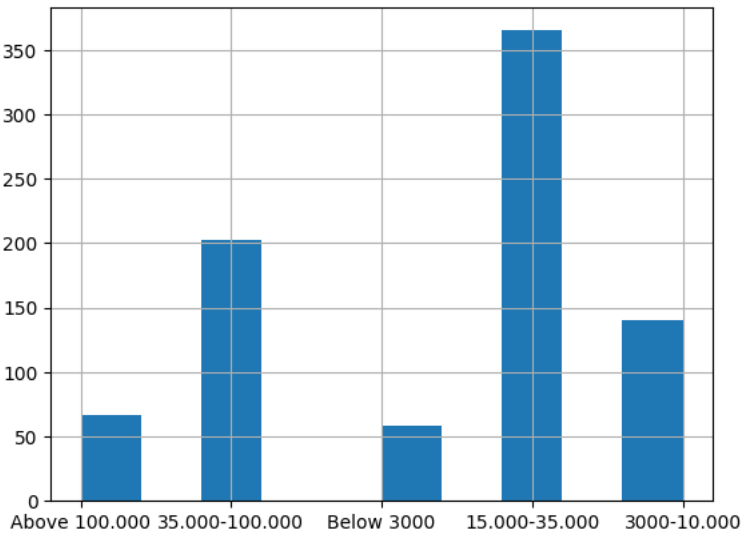
```
df2[['Total operating revenue', 'Rooftop Size factory 1 (m2)', 'No of Employees']].describe().applymap('{:,.2f}'.format)
```

```
Total operating revenue  Rooftop Size factory 1 (m2)  No of Employees
```

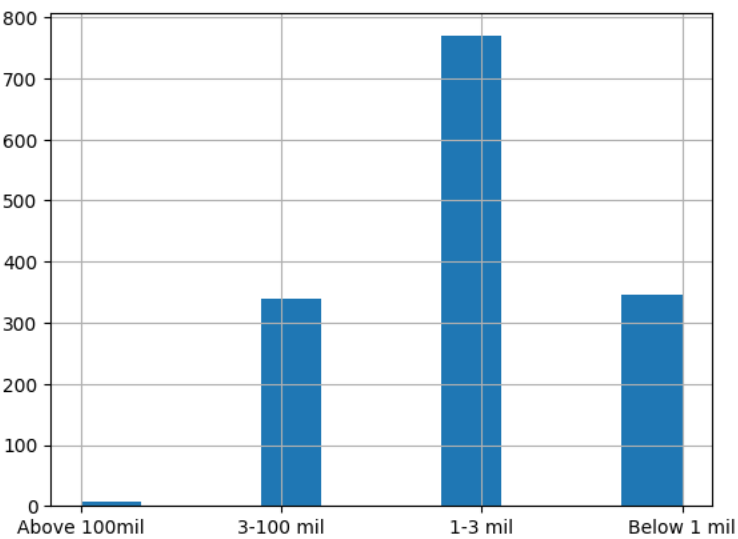
```
df2['No of Employees Range'] = pd.cut(x=df2['No of Employees'], bins=[1, 160, 600, 1576, 15000, 62000],
                                      labels=['Below 160', '161-600', '601-1576', '1576-15000', 'Above 15000'])
df2['Rooftop Size'] = pd.cut(x=df2['Rooftop Size factory 1 (m2)'], bins=[1, 3000, 10000, 35000, 100000, 880017],
                                      labels=['Below 3000', '3000-10.000', '15.000-35.000', '35.000-100.000', 'Above 100.000'])
df2['Revenue'] = pd.cut(x=df2['Total operating revenue'], bins=[1, 1000000, 3000000, 100000000, 500000000],
                                      labels=['Below 1 mil', '1-3 mil', '3-100 mil', 'Above 100mil'])
```

```
import matplotlib.pyplot as plt
```

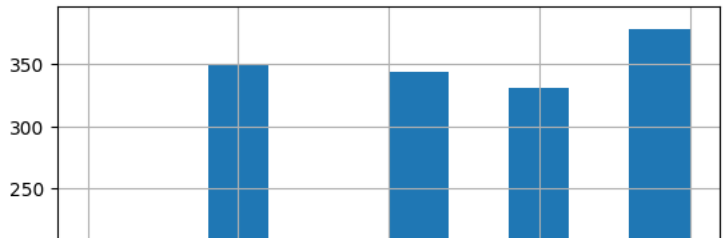
```
df2['Rooftop Size'].hist();
```




```
df2['Revenue'].hist();
```




```
df2['No of Employees Range'].hist();
```



3. Export data

```
df2.to_csv("Data_BI_Practise_CME.csv", index=False, encoding="utf-8-sig")
```

```
def city_state_country(row):
    from geopy.geocoders import Nominatim
    geolocator = Nominatim(user_agent="geoapiExercises")
    coord = f"{row['lat']}, {row['lon']}"
    location = geolocator.reverse(coord, exactly_one=True)
    address = location.raw['address']
    city = address.get('city', '')
    state = address.get('state', '')
    country = address.get('country', '')
    row['city'] = city
    row['state'] = state
    row['country'] = country
    return row

# df = df.apply(city_state_country, axis=1)
# print(df)


# df2['company'] = df2['lat-lon'].apply(getAddress)
df2['province'] = df2['lat-lon'].apply(getAddress)
# df2['lat-lon']
```

```
WARNING:urllib3.connectionpool:Retrying (Retry(total=1, connect=None, read=None, redirect=None, status=None)) after connection broken b
WARNING:urllib3.connectionpool:Retrying (Retry(total=1, connect=None, read=None, redirect=None, status=None)) after connection broken b
WARNING:urllib3.connectionpool:Retrying (Retry(total=1, connect=None, read=None, redirect=None, status=None)) after connection broken b
WARNING:urllib3.connectionpool:Retrying (Retry(total=1, connect=None, read=None, redirect=None, status=None)) after connection broken b
```

```
df2.head(3)
```

Insights

Below are key notes.

- There are 41.94% clients of 2.1% existing clients from other products have not yet existing solar rooftop. This is a source of potential customers for the company to cross-sell solar products.
- A big potential customers representing about 97.9% of the market are not yet existing client of the company. We need to have a strategy to approach and introduce the company's products to these customers. 5.2% of that already have solar rooftop, we can approach for maintenance service accessories products. 65.2% of that have no existing solar rooftop are mainly in Southern Region.